Verificación y Validación de Modelos de Simulación
¿Construimos el modelo correctamente y construimos el modelo correcto?

T3.2 · Modelado de Sistemas bajo Incertidumbre

Alejandra Tabares Pozos

Universidad de los Andes · Ingeniería Industrial

2026

Estructura de las dos sesiones

Sesión 1 — Verificación

¿El código implementa el modelo conceptual?

- Distinción formal V&V (marco de Sargent).
- *Debugging* con aserciones e invariantes.
- Trazas de eventos y verificación contra cálculo manual.
- Pruebas unitarias (patrón AAA).
- Verificación contra resultado analítico ($M/M/1$).
- **Ejemplo 1** — verificación completa de un $M/M/1$ básico.

Sesión 2 — Validación

¿El modelo conceptual representa al sistema real?

- Niveles de validación (Sargent).
- *Face validity*: protocolo y cuestionario.
- Validación de supuestos: χ^2 y Kolmogorov–Smirnov.
- Calibración y división de muestra.
- Validación I/O con prueba t pareada.
- Test de Turing (Schruben).
- **Ejemplo 2** — bug de parametrización detectado.
- **Ejemplo 3** — celda $M/E_2/1$ + sensibilidad.

Idea central

Validar y verificar no son trámites finales: son las herramientas con las que un modelo gana **credibilidad**. Sin V&V, ningún tomador de decisiones debería actuar sobre los resultados de la simulación.

Las dos preguntas: validación y verificación

Validación

*¿Estamos construyendo el modelo **correcto**?*

Proceso de establecer que el **modelo conceptual** es una representación adecuada del sistema real *para los propósitos del estudio*.

$\Rightarrow$ Compara **modelo** contra **realidad**.

Verificación

*¿Estamos construyendo el modelo **correctamente**?*

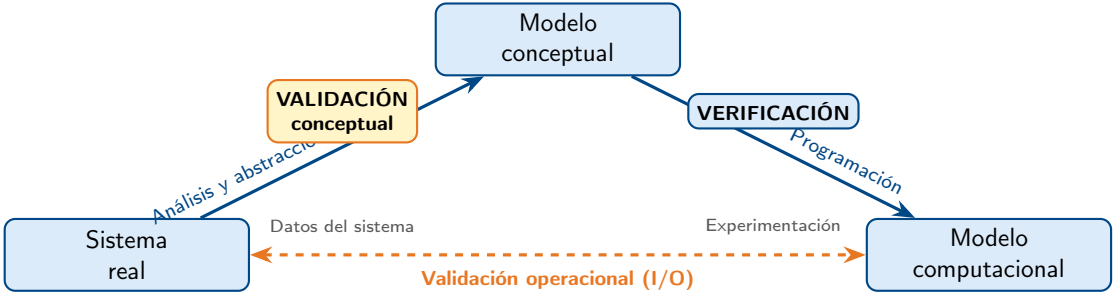
Proceso de establecer que la **implementación computacional** corresponde fielmente al **modelo conceptual**.

$\Rightarrow$ Compara **código** contra **modelo conceptual**.

Orden conceptual vs. orden de ejecución

Conceptualmente, la validación es primaria: si el modelo conceptual no representa al sistema, no importa qué tan bien programado esté el código. **En la práctica**, se verifica el código a medida que se programa y se valida el modelo de manera *iterativa* contra la realidad.

Ubicación de V&V en el ciclo de modelado (Sargent)



La **validación** actúa entre el sistema real y el modelo (conceptual y operacional). La **verificación** actúa exclusivamente entre el modelo conceptual y su implementación.
Adaptado de Sargent (2013) y Banks et al. (2010).

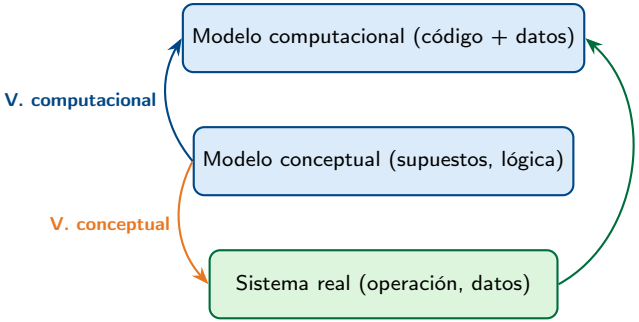
Comparación lado a lado

	Validación	Verificación
Pregunta	¿El modelo correcto?	¿Construido correctamente?
Compara	Modelo vs. realidad	Código vs. modelo conceptual
Naturaleza	Estadística y de modelado	Ingeniería de software
Errores que detecta	Supuestos inadecuados, distribuciones mal ajustadas, lógica de negocio incorrecta	Bugs, errores lógicos, errores numéricos
Herramientas	Pruebas estadísticas, expertos del dominio, datos históricos	Debugger, trazas, pruebas unitarias
¿Cuándo se hace?	Antes, durante y después de modelar	Durante y después de programar

Moraleja

Un modelo puede estar **verificado** (sin bugs) pero no **validado** (con supuestos incorrectos), y viceversa. Ambos procesos son necesarios; ninguno sustituye al otro.

Marco de Sargent: tres tipos de validez



1. **Validez conceptual**
   ¿Es razonable la teoría y la estructura del modelo? Supuestos, distribuciones (T3.1), lógica de eventos.
2. **Validez computacional**
   ¿La implementación reproduce el modelo conceptual? Es la verificación.
3. **Validez operacional**
   ¿Las salidas del modelo coinciden con las del sistema real dentro de un margen aceptable?

Validez de datos cruza las tres capas.

V&V es un proceso continuo

Cada modificación al modelo (nueva distribución, nuevo evento, nuevo parámetro) requiere re-verificación y potencialmente re-validación. Un modelo validado para un rango de operación puede no serlo fuera de él.

Sesión 1

Verificación del modelo computacional

Debugging, trazas y pruebas unitarias

Panorama de técnicas de verificación

Cuatro categorías (Banks et al., cap. 10; Law, cap. 5):

1. **Inspección estática del código**
   - Revisión por pares (*code review*, *walkthroughs*).
   - Análisis estático automatizado (*linters*).
2. **Inspección dinámica de la ejecución** ***(debugging y trazas)***
   - Depuradores interactivos, *logging*, *breakpoints*, *watch*.
   - Trazas de eventos y de estado.
3. **Pruebas formales** ***(pruebas unitarias)***
   - Pruebas unitarias (*unit tests*).
   - Pruebas de integración.
   - Pruebas de regresión.
4. **Pruebas de coherencia con resultados conocidos**
   - Comparación con soluciones analíticas ($M/M/1$, $M/M/c$, P-K).
   - Casos límite (degeneración, continuidad).

Filosofía operativa

Una aserción violada *detiene* la simulación y *señala* dónde está el error. Es radicalmente más eficiente que mirar reportes de salida tratando de adivinar qué celda contiene el bug.

Debugging: definición y herramientas formales

Debugging

Proceso sistemático de identificar, localizar y corregir defectos (*bugs*) en el código. Tres fases: *(i)* **detección** (síntoma observable), *(ii)* **localización** (causa raíz), *(iii)* **corrección** (solución verificada).

Herramientas formales que usaremos:

Técnica	Mecanismo	Cuándo se usa
Aserciones (`assert`)	Detiene la ejecución si una invariante se viola	Verificar pre/post-condiciones e invariantes
Logging estructurado	Registra eventos con marca de tiempo y nivel	Ejecuciones largas, reproducción a posteriori
Breakpoints	Pausa en líneas o condiciones específicas	Análisis interactivo del estado
Inspección de variables (*watch*)	Vigila el cambio de un valor	Detectar cuándo una variable toma valor inesperado
Reproducibilidad por semilla	Fija la semilla aleatoria	Aislar bugs intermitentes

Debugging con aserciones — ejemplo $M/M/1$

Invariantes que debe respetar el simulador:

$$N_{\text{sis}}(t) = N_q(t) + N_s(t), \quad N \geq 0, \quad t_{\text{ev},k+1} \geq t_{\text{ev},k}$$

```python
def manejar_salida(self, t):
    # Pre-condiciones (deben cumplirse SIEMPRE)
    assert t >= self.reloj, f"Tiempo retrocede: {t} < {self.reloj}"
    assert self.n_servicio == 1, "Salida sin cliente en servicio"
    assert self.n_cola >= 0, f"Cola negativa: {self.n_cola}"

    self.reloj = t
    self.servidos += 1
    self.n_servicio = 0

    if self.n_cola > 0:                          # Toma el siguiente
        self.n_cola -= 1
        self.n_servicio = 1
        self.programar_salida(t + self.gen_servicio())

    # Post-condicion: invariante de conservacion
    assert self.llegados == self.servidos + self.n_cola + self.n_servicio, \
           "Violacion de conservacion de clientes"
```

Logging estructurado: bugs sutiles

```python
import logging
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s [%(levelname)s] %(message)s')

def manejar_llegada(self, t):
    self.reloj = t
    self.llegados += 1
    logging.debug(f"LLEGADA t={t:.4f} cliente={self.llegados} cola={self.n_cola}")

    if self.n_servicio == 0:
        self.n_servicio = 1
        ts = self.gen_servicio()
        self.programar_salida(t + ts)
        logging.info(f"INICIO SERV. t={t:.4f} servicio={ts:.4f}")
    else:
        self.n_cola += 1
        if self.n_cola > 100:
            logging.warning(f"COLA GRANDE: {self.n_cola} en t={t:.4f}")

    self.programar_llegada(t + self.gen_interarribo())
```

Por qué funciona

Reconstruye la historia completa de la corrida y permite filtrar por nivel (`DEBUG`, `INFO`, `WARNING`). Detecta condiciones anómalas *sin detener* la simulación.

Trazas: definición formal

Traza

Una **traza** es la secuencia ordenada de *tuplas de estado* del sistema simulado, registradas en cada cambio de estado:

$$\mathcal{T} = \{(t_k, e_k, \mathrm{s}_k) : k = 1, 2, \ldots, K\}$$

donde t_k es el reloj de simulación, e_k el evento ejecutado y s_k el vector de estado del sistema inmediatamente después del evento.

Tres usos formales de la traza para verificación:

1. **Inspección manual** (*eyeballing*): primeras 30–50 líneas para revisar lógica.
2. **Verificación de invariantes**: chequear automáticamente propiedades sobre $\mathcal{T}$ (conservación, no-negatividad, monotonía).
3. **Comparación con cálculo a mano**: ejecutar el modelo con *inputs* prefijados (no aleatorios) y verificar la traza contra resultados calculados manualmente.

Traza $M/M/1$ con *inputs* prefijados

Inputs deterministas (un solo cajero):

- Tiempos entre llegadas: 1,2; 0,5; 1,8; 0,6; 2,1
- Tiempos de servicio: 0,8; 1,5; 0,4; 1,1; 0,7

Traza esperada (calculada a mano):

```
t        Evento    Cliente  N_cola  N_serv  Acumulado
1.200    LLEG      C1       0       1       L=1
2.000    SAL       C1       0       0       (servicio = 0.8)
1.700    LLEG      C2       0       1       (1.2+0.5)
3.200    SAL       C2       0       0       (servicio = 1.5)
3.500    LLEG      C3       0       1       (1.7+1.8)
3.900    SAL       C3       0       0       (servicio = 0.4)
4.100    LLEG      C4       0       1       (3.5+0.6)
5.200    SAL       C4       0       0       (servicio = 1.1)
6.200    LLEG      C5       0       1
6.900    SAL       C5       0       0
```

Regla de oro

La traza producida por el simulador debe ser *idéntica* línea por línea. Cualquier diferencia $\Rightarrow$ **bug localizado en ese evento**.

Verificación automática de invariantes sobre la traza

Invariantes formales que se chequean al recorrer $\mathcal{T}$:

Invariante	**Expresión formal**
Monotonicidad del reloj	$t_{k+1} \geq t_k \ \ \forall k$
No-negatividad de la cola	$N_{q,k} \geq 0 \ \ \forall k$
Conservación de clientes	$A_k = D_k + N_{\mathbf{sis},k}$ (arrivals = departures + WIP)
Capacidad del servidor	$0 \leq N_{\mathbf{serv},k} \leq c$
FCFS (orden de salida)	si $a_i < a_j \Rightarrow d_i \leq d_j$
Trabajo conservado (Little)	$L = \lambda W$ en estado estacionario (con tolerancia)

Implementación

Un *post-procesador* lee la traza y verifica cada invariante. Si alguna falla $\Rightarrow$ línea exacta donde aparece el bug. El costo computacional es *lineal* en el número de eventos, despreciable frente al de la simulación.

Pruebas unitarias: definición y patrón AAA

Prueba unitaria

Programa que ejecuta una unidad mínima de código (función, método o módulo) con *inputs* controlados y verifica automáticamente que el *output* coincida con el valor esperado.

Patrón *Arrange–Act–Assert* (AAA):

1. **Arrange**: preparar el escenario (datos, semilla, instancia).
2. **Act**: ejecutar la unidad bajo prueba.
3. **Assert**: comparar el resultado con el valor esperado.

Tres categorías de pruebas para un simulador:

- **Pruebas de límite**: $\lambda = 0$, $\mu \to \infty$, capacidad $\to 0$.
- **Pruebas de invariantes**: conservación, no-negatividad, FCFS.
- **Pruebas contra valor analítico**: $M/M/1$, $M/M/c$, $M/M/1/K$, $M/G/1$ (P-K).

Pruebas unitarias para un $M/M/1$ (pytest)

```python
import pytest
from simulador import SimuladorMM1

def test_sin_llegadas_servidor_ocioso():
    sim = SimuladorMM1(lam=0.0, mu=1.0, semilla=42)
    sim.correr(t_max=1000)
    assert sim.servidos == 0
    assert sim.utilizacion() == 0.0

def test_conservacion_clientes():
    sim = SimuladorMM1(lam=0.8, mu=1.0, semilla=42)
    sim.correr(t_max=1000)
    assert sim.llegados == sim.servidos + sim.en_sistema()

def test_estado_estacionario_vs_analitico():
    """L = rho/(1-rho) para M/M/1; rho=0.5 -> L=1.0"""
    sim = SimuladorMM1(lam=0.5, mu=1.0, semilla=42)
    sim.correr(t_max=200_000, t_warmup=20_000)
    assert sim.L_promedio() == pytest.approx(1.0, abs=0.05)

def test_rho_mayor_uno_cola_crece():
    sim = SimuladorMM1(lam=1.2, mu=1.0, semilla=42)
    sim.correr(t_max=10_000)
    assert sim.cola_max() > 100    # Sistema inestable
```

$M/M/1$ como banco de pruebas — verificación contra teoría

Por qué $M/M/1$: solución cerrada conocida; permite comprobar el simulador con alta precisión.

Fórmulas analíticas

Con $\rho = \lambda/\mu < 1$:

$$L = \frac{\rho}{1-\rho}, \quad L_q = \frac{\rho^2}{1-\rho}, \quad W = \frac{1}{\mu-\lambda}, \quad W_q = \frac{\rho}{\mu-\lambda}$$

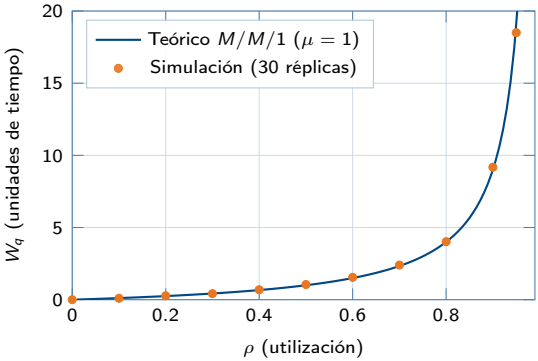
Tabla de verificación ($\lambda = 0{,}8$; $\mu = 1{,}0$; $\rho = 0{,}8$; 30 réplicas; $T = 10\,000$; *warm-up* $= 1\,000$):

Métrica	Analítico	Simulación	IC 95 %	¿Aprueba?
L	4,000	4,018	[3,91; 4,13]	✓
L_q	3,200	3,215	[3,10; 3,33]	✓
W	5,000	5,027	[4,88; 5,17]	✓
W_q	4,000	4,020	[3,88; 4,16]	✓

Criterio formal

Verificación aceptada si el valor analítico cae dentro del IC al 95 % para **todas** las métricas.

W_q vs ρ: prueba de coherencia teórica



Lectura:

- Los puntos siguen la curva en todo el rango.
- Comportamiento asintótico correcto cuando $\rho \to 1$ ($W_q \to \infty$).
- No hay desviaciones sistemáticas (sesgo).

Si la curva no diverge

Hay un truncamiento o un límite "invisible" en el código (cola con capacidad accidentalmente fijada).

Ejemplo 1 — Verificación completa de un $M/M/1$

Sistema

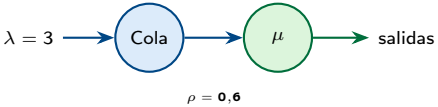
Cola $M/M/1$ con $\lambda = 3$ clientes/min y $\mu = 5$ clientes/min. Antes de usar el modelo para análisis, se aplica el catálogo completo de pruebas de verificación.

Valores analíticos de referencia:

$$\rho = \frac{\lambda}{\mu} = 0{,}60$$

$$W_q = \frac{\rho}{\mu(1-\rho)} = \frac{0{,}6}{5 \times 0{,}4} = 0{,}30 \text{ min}$$

$$L_q = \frac{\rho^2}{1-\rho} = \frac{0{,}36}{0{,}4} = 0{,}90 \text{ clientes}$$

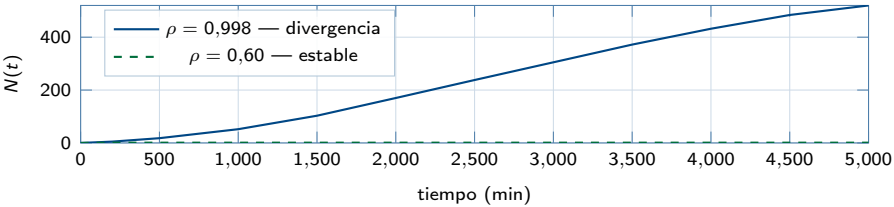


Pruebas a realizar:

- Degenerada $\lambda = 0$
- Conservación
- Traza manual
- Tráfico extremo $\rho \to 1$

Ejemplo 1 — Resultados de las pruebas

Prueba	Esperado	Observado
Degenerada $\lambda = 0,\ T = 1000$	$N(t) \equiv 0,\ \rho = 0,\ W_q = 0$	Exactamente 0 en todas las métricas ✓
Conservación, $T = 1000$	$\sum_{\text{lleg}} = \sum_{\text{sal}} + N(T)$	$3\,012 = 3\,012 + 0$ ✓
Traza primeros 6 eventos	Coincide con cálculo manual	Tiempos exactos al 4to decimal ✓
Tráfico extremo, $\rho = 0{,}998,\ T = 5000$	L_q crece sin cota	$L_q > 500$ al final ✓



Verificación cerrada

Todas las pruebas pasan. El código implementa fielmente el modelo conceptual $M/M/1$. Procedemos a *validar* la salida.

Lo que hemos hecho

Hemos cubierto las tres familias de técnicas formales para verificar el modelo computacional:

1. ***Debugging*** con aserciones de invariantes y *logging* estructurado.
2. **Trazas** con verificación contra cálculo manual y chequeo automático de invariantes.
3. **Pruebas unitarias** con patrón AAA, incluyendo pruebas contra resultados analíticos.

Próxima sesión

Suponer ahora que el modelo está *verificado* (el código hace lo que dice el modelo conceptual).
¿Pero el modelo conceptual representa al sistema real?
Esa es la pregunta de la **validación**.

Sesión 2

Calibración y Validación
del modelo conceptual

Face validity · Validación de supuestos · Calibración
Input–Output · Test de Turing · Sensibilidad

Niveles de validación (Sargent, 2013)

Validación operacional / Input–Output: salidas del modelo $\approx$ salidas reales

Validación conceptual: estructura y supuestos del modelo $\approx$ realidad

Validación de datos: distribuciones de *inputs* ajustadas correctamente

Face validity: el modelo es razonable a juicio de expertos

Filosofía de fondo

Ningún test individual "valida" el modelo. La validación es una **acumulación de evidencia** (Balci, 1997). Cada técnica aporta un argumento. La confianza en el modelo crece con la cantidad y diversidad de pruebas superadas.

Face Validity: protocolo formal

Face validity

Evaluación cualitativa del modelo por parte de **expertos del dominio** (no necesariamente modeladores) sobre si la estructura, los supuestos y el comportamiento del modelo *aparentan* ser razonables.

Protocolo de *face validity* (Sargent)

1. Identificar 3–5 expertos del sistema real (operadores, gerentes, ingenieros).
2. Presentarles, por separado:
   - Diagrama de flujo del modelo conceptual.
   - Lista explícita de supuestos.
   - Animación o resultados gráficos del modelo.
3. Aplicar un cuestionario estructurado.
4. Documentar discrepancias y **re-modelar** antes de continuar.

Ejemplo (call center): el supervisor observa la animación. Detecta que el modelo nunca refleja "rebalanceos de agentes a otra línea". Esto cambia la *estructura* del modelo, no solo sus parámetros.

Cuestionario estructurado para *face validity*

Plantilla (puntuación 1 = totalmente en desacuerdo; 5 = totalmente de acuerdo):

Ítem	**Puntaje**
1. Las entidades modeladas (clientes, agentes, recursos) corresponden a las que existen en el sistema real.	____
2. Los eventos modelados cubren los procesos relevantes del sistema.	____
3. La política de operación implementada coincide con la real.	____
4. Los volúmenes y tiempos observados en la animación parecen realistas.	____
5. Los casos extremos (picos, fallas, ausencias) están razonablemente representados.	____
6. Las exclusiones del modelo (qué *no* se modela) están justificadas.	____

Criterio

Promedio $\geq 4{,}0$ **y** ningún ítem por debajo de 3 (todos los expertos). De lo contrario, revisar el modelo. *Inspirado en Sargent (2013) y Balci (1997).*

Validación de supuestos: estructurales y de datos

Dos categorías formales (Law, cap. 5):

Supuestos estructurales

Sobre la *lógica* del sistema:

- Disciplina de cola (FIFO, prioridad).
- Independencia entre recursos.
- Bloqueo, abandonos, reintentos.
- Política de despacho.

Validación: observación directa, entrevistas con operadores, manuales de operación.

Supuestos sobre datos

Sobre las *distribuciones* de los *inputs*:

- Tiempos entre llegadas $\sim \mathrm{Exp}(\lambda)$.
- Tiempos de servicio $\sim \mathrm{LogN}(\mu, \sigma)$.
- Independencia entre observaciones.
- Estacionariedad.

Validación: pruebas estadísticas formales (*goodness-of-fit*).

Lazo con T3.1

Las pruebas χ^2, Kolmogorov–Smirnov y Anderson–Darling vistas en T3.1 se usan ahora con un **propósito explícito de validación**: aceptar o rechazar la hipótesis estructural sobre los datos del sistema real.

Validación estructural — sucursal bancaria

Sistema: sucursal con 3 cajeros, una sola cola común.

Supuestos del modelo conceptual:

1. Cola única, FIFO.
2. Cuando un cajero queda libre, toma al primero de la cola.
3. Servicios independientes entre cajeros.
4. No hay abandonos.

Procedimiento de validación estructural:

Supuesto	Cómo validar
Cola única FIFO	Observar 2 h en sucursal: ¿hay corte de fila? ¿saltos?
Cajero libre toma al primero	Observar transiciones: ¿algún cajero llama por número?
Servicios independientes	Comparar tiempos de servicio entre cajeros (ANOVA).
Sin abandonos	Conteo de clientes que entran vs. que salen sin servicio.

Hallazgo típico

Se detecta un 4 % de abandonos $\Rightarrow$ **el supuesto 4 falla**, hay que modelar abandonos. Esto no se descubre con pruebas estadísticas; se descubre *mirando el sistema*.

Validación de datos: prueba χ^2 de bondad de ajuste

Prueba χ^2

Hipótesis: H_0: los datos provienen de la distribución F_0 propuesta.
Estadístico:

$$\chi_0^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

O_i = frecuencia observada en la clase i; $E_i = n\,p_i$ = frecuencia esperada bajo F_0.
Distribución: $\chi_0^2 \sim \chi^2_{k-s-1}$ bajo H_0, con s = parámetros estimados de los datos.
Decisión: rechazar H_0 si $\chi_0^2 > \chi^2_{\alpha,\,k-s-1}$.

Reglas prácticas

(i) cada $E_i \geq 5$; (ii) usar al menos $k = 5$ clases; (iii) preferir clases de *probabilidad igual* bajo F_0.

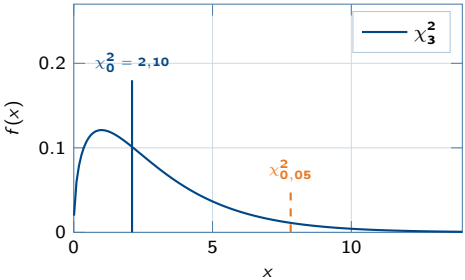
Datos: $n = 100$ tiempos entre llegadas a un cajero. Media muestral $\bar{X} = 2{,}50$ min $\Rightarrow \hat{\lambda} = 1/\bar{X} = 0{,}40$.

Construcción de clases con probabilidades iguales ($k = 5$, $p_i = 0{,}20$):

i	Intervalo	p_i	E_i	O_i	$\frac{(O-E)^2}{E}$
1	$[0;\ 0{,}558)$	0.20	20	23	0.450
2	$[0{,}558;\ 1{,}277)$	0.20	20	17	0.450
3	$[1{,}277;\ 2{,}291)$	0.20	20	18	0.200
4	$[2{,}291;\ 4{,}023)$	0.20	20	24	0.800
5	$[4{,}023;\ \infty)$	0.20	20	18	0.200
				$\chi^2_0 =$	**2.100**

Con $\alpha = 0{,}05$, $k = 5$, $s = 1$: $\chi^2_{0{,}05,\,3} = 7{,}815$. Como $2{,}10 < 7{,}815$ **no se rechaza** H_0.

Densidad χ^2_3 y región de rechazo



El estadístico cae lejos de la región crítica.

Conclusión

El supuesto exponencial es razonable para los interarribos a este nivel de evidencia.

Validación de datos: prueba de Kolmogorov–Smirnov

Prueba KS

Estadístico:

$$D_n = \sup_x \left| F_n(x) - F_0(x) \right|$$

F_n = CDF empírica; F_0 = CDF teórica propuesta.
Decisión: rechazar H_0 si $D_n > D_{\alpha,n}$ (valor crítico).

Ventajas sobre χ^2:

- No requiere agrupación en clases (válido para muestras pequeñas).
- Más potente para distribuciones continuas.

Limitación

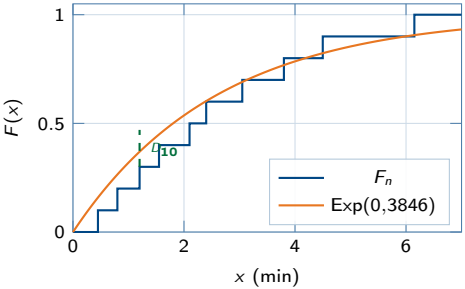
Cuando los parámetros se estiman de los datos (Exp, Normal, Weibull) los valores críticos tabulados son **conservadores**. Hay tablas corregidas: **Lilliefors** (normal), **KS para exponencial con $\hat{\lambda}$**.

KS trabajado — muestra pequeña de tiempos de servicio

Datos: $n = 10$ tiempos de servicio (min): {0,45; 0,80; 1,20; 1,55; 2,10; 2,40; 3,05; 3,80; 4,50; 6,15}.
$\bar{X} = 2{,}60 \Rightarrow \hat{\lambda} = 0{,}3846$. H_0: Exp($\hat{\lambda}$).

i	$x_{(i)}$	F_n	F_0	$\lvert F_n - F_0 \rvert$	$\lvert F_n^- - F_0 \rvert$
1	0.45	0.10	0.1585	0.0585	0.1585
2	0.80	0.20	0.2659	0.0659	0.1659
3	1.20	0.30	0.3700	0.0700	0.1700
4	1.55	0.40	0.4493	0.0493	0.1493
5	2.10	0.50	0.5538	0.0538	0.1538
6	2.40	0.60	0.6020	0.0020	0.1020
7	3.05	0.70	0.6904	0.0096	0.0904
8	3.80	0.80	0.7676	0.0324	0.0676
9	4.50	0.90	0.8254	0.0746	0.0254
10	6.15	1.00	0.9062	0.0938	0.0062

ECDF F_n vs. teórica F_0



$D_{10} = 0{,}1700$. Crítico (Lilliefors-exp):
$D_{0{,}05;\,10} \approx 0{,}338$.
$0{,}170 < 0{,}338 \Rightarrow$ no se rechaza H_0.

Calibración: definición y proceso

Calibración

Ajuste iterativo de los **parámetros** y, ocasionalmente, de la **estructura** del modelo, hasta que sus salidas reproducen razonablemente las del sistema real bajo condiciones conocidas.

Calibración $\neq$ Validación:

- **Calibrar:** *ajustar* para que el modelo se parezca a los datos.
- **Validar:** *probar* el modelo contra datos **independientes** a los usados para calibrar.

Procedimiento de división de muestra

1. Particionar los datos en **conjunto de calibración** (70 %) y **conjunto de validación** (30 %).
2. Ajustar parámetros sobre el conjunto de calibración (MLE).
3. Correr la simulación y comparar con el conjunto de validación.
4. Si el desempeño en validación es pobre, revisar *estructura* (no solo parámetros) y repetir.

Ejemplo de calibración — tiempos de servicio del cajero

Sistema: sucursal bancaria. $n = 600$ observaciones de tiempos de servicio.

Paso 1. Partición:
Calibración: 420 obs (70 %). Validación: 180 obs (30 %).

Paso 2. Calibración por máxima verosimilitud:
Suponemos $\mathrm{LogN}(\mu, \sigma)$. MLE sobre el conjunto de calibración:
$\hat{\mu} = 0{,}812$, $\hat{\sigma} = 0{,}443$ ($\bar{X}_{\text{cal}} = 2{,}48$ min).

Paso 3. Validación cruzada: prueba KS de los 180 datos de validación contra $\mathrm{LogN}(0{,}812;\ 0{,}443)$.
$D_{180} = 0{,}061$. Crítico Lilliefors $= 0{,}066$ a $\alpha = 0{,}05$.
$0{,}061 < 0{,}066 \Rightarrow$ ajuste valida en datos no usados.

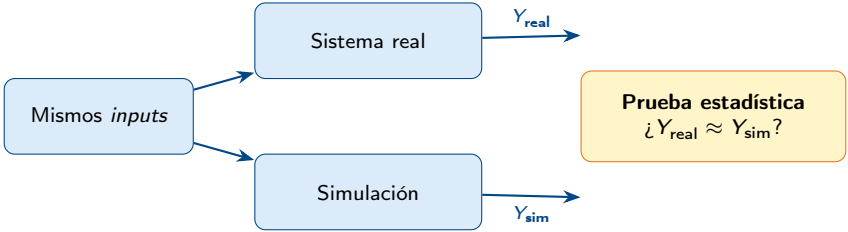
Advertencia

Si la prueba KS hubiera fallado en la *misma* muestra usada para calibrar, no significaría nada (autocomplacencia). La prueba en **datos independientes** es lo que da validez.

Validación Input–Output: idea general

Validación I/O

Comparar las **salidas** del modelo de simulación con las **salidas observadas** del sistema real, cuando ambos operan bajo **las mismas condiciones de entrada**.



Pruebas estadísticas comunes:

- Prueba t **pareada** (cuando hay correspondencia uno-a-uno entre días/horas).
- Prueba t de **dos muestras** (cuando no la hay).
- IC para la diferencia de medias: si $0 \in IC \Rightarrow$ no hay evidencia de diferencia.

I/O con prueba t pareada — definición formal

Prueba t pareada

Sean $Y_i^{\text{real}}, Y_i^{\text{sim}}$ con $i = 1, \ldots, n$ pares (mismo día/hora). Definir $D_i = Y_i^{\text{real}} - Y_i^{\text{sim}}$, $\bar{D}$ y S_D media y desviación muestrales.

$$T = \frac{\bar{D}}{S_D/\sqrt{n}} \sim t_{n-1} \quad \text{bajo} \quad H_0 : \mu_D = 0$$

IC del $100(1-\alpha)\,\%$ para μ_D:

$$\bar{D} \pm t_{\alpha/2,\, n-1} \frac{S_D}{\sqrt{n}}$$

Por qué pareada

Controla la variabilidad común a real y simulación (mismo día = mismas condiciones), aumentando la potencia frente a la prueba de dos muestras.

I/O trabajado — tiempo medio de espera diario, $n = 10$ días

Sistema: cola de servicio al cliente. Para cada día se mide el tiempo medio de espera real y se ejecuta una réplica con los *inputs* de ese día.

Día i	1	2	3	4	5	6	7	8	9	10
$Y_i^{\mathbf{real}}$ (min)	4.2	5.1	3.8	6.0	4.7	5.5	3.9	4.4	5.8	4.6
$Y_i^{\mathbf{sim}}$ (min)	4.5	4.8	3.6	5.7	4.9	5.2	4.1	4.2	5.5	4.8
D_i	-0.3	0.3	0.2	0.3	-0.2	0.3	-0.2	0.2	0.3	-0.2

Cálculos:

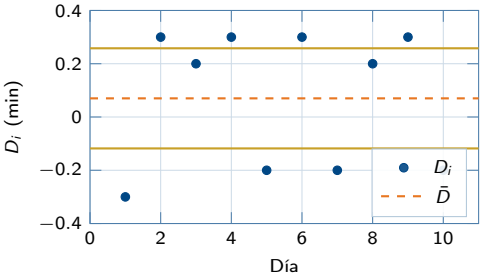
$$\bar{D} = 0{,}07, \quad S_D = 0{,}2627, \quad n = 10$$

$$T = \frac{0{,}07}{0{,}2627/\sqrt{10}} = \frac{0{,}07}{0{,}0831} = 0{,}843$$

Con $\alpha = 0{,}05$, $t_{0{,}025;9} = 2{,}262$: $|T| = 0{,}843 < 2{,}262$
$\Rightarrow$ no se rechaza H_0.

IC 95 % para μ_D:
$0{,}07 \pm 2{,}262 \cdot 0{,}0831 = [-0{,}118;\ 0{,}258]$.
Como $0 \in IC$, **no hay evidencia de diferencia**.

Diferencias diarias D_i y su IC



El IC contiene 0.

Significancia estadística vs. operacional

Caso opuesto: con n muy grande, casi cualquier diferencia se vuelve estadísticamente significativa. ¿Significa eso que el modelo no es válido?

Criterio adicional: tolerancia operacional δ^*

Definir *a priori* un umbral δ^* de diferencia que es **operacionalmente irrelevante**.

- $IC \subset (-\delta^*, \delta^*) \Rightarrow$ **validación fuerte** (diferencia indistinguible de irrelevante).
- $0 \in IC$ pero $IC \not\subset (-\delta^*, \delta^*) \Rightarrow$ **validación débil** (no se detecta diferencia, pero podría haberla relevante; aumentar n).
- $0 \notin IC$ y $IC \subset (-\delta^*, \delta^*) \Rightarrow$ **diferencia detectada pero irrelevante**.
- $0 \notin IC$ y $IC \not\subset (-\delta^*, \delta^*) \Rightarrow$ **revisar el modelo**.

El criterio que separa al modelador del estudiante

Esta clasificación es la que separa al modelador entrenado del estudiante que mira solo el p-valor (Law, cap. 5).

Test de Turing para simulación (Schruben, 1980)

Test de Turing

Se le presentan a un experto del sistema K reportes mezclados, algunos generados por el sistema real y otros por la simulación. El experto debe clasificar cada reporte como "real" o "simulado". Si el experto **no puede distinguir** mejor que el azar, el modelo se considera válido.

Procedimiento de Schruben

1. Generar K_R reportes del sistema real y K_S reportes de la simulación, con *el mismo formato*.
2. Mezclarlos aleatoriamente; etiquetar internamente.
3. Presentar al experto, sin decirle cuáles son cuáles.
4. El experto clasifica cada reporte.
5. Construir tabla de aciertos $\rightarrow$ prueba estadística.

Test de Turing: análisis estadístico de los aciertos

Diseño $K_R = K_S = K$ con $K = 10$ reportes de cada tipo (total 20).

Tabla de contingencia (clasificación del experto):

	Clasif. "Real"	Clasif. "Sim."	Total
Realmente real	6	4	10
Realmente simulado	5	5	10
Total	11	9	20

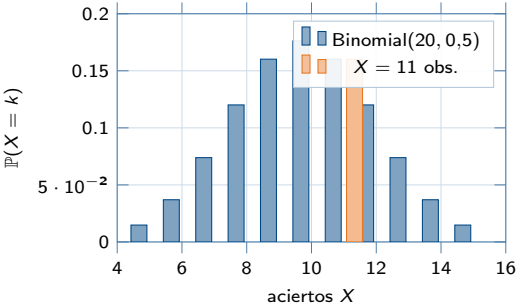
Aciertos: $6 + 5 = 11$ de 20 $\Rightarrow$ tasa $= 0{,}55$.

Prueba binomial: H_0: $p = 0{,}5$ (el experto adivina).

$$\mathbb{P}(X \geq 11 \mid n = 20,\ p = 0{,}5) = 0{,}412$$

p-valor alto $\Rightarrow$ no se rechaza H_0: el experto no logra distinguir mejor que el azar.

Distribución de aciertos bajo H_0 ($n = 20$, $p = 0{,}5$)



Conclusión

Las salidas simuladas son **indistinguibles**, en juicio experto, de las reales.

Buenas prácticas del Test de Turing

- Usar formato **idéntico** en reportes reales y simulados (incluso números de decimales y orden de filas). Cualquier diferencia de formato es una pista que invalida el test.
- Usar **varios expertos independientes**, no uno solo.
- Pedirle al experto que **justifique** su clasificación: las razones revelan qué aspectos del modelo son débiles, incluso cuando el test pasa.
- Repetir el test con K moderadamente grande (al menos 10 de cada tipo) para tener potencia estadística.
- No es un test único: combinar con I/O y validación de supuestos.

Cuidado

Un test de Turing aprobado **no** significa que el modelo sea correcto, solo que es *indistinguible* en los *outputs* examinados. El experto puede pasar por alto patrones sutiles.

Ejemplo 2 — Validación del $M/M/1$ y un bug sutil

Sistema y dos versiones del código

Mismo sistema $M/M/1$ con $\lambda = 3$, $\mu = 5$ que el Ejemplo 1. Comparamos:

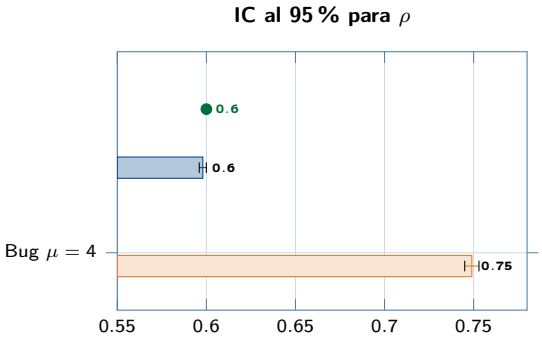
- **Versión correcta:** $\mu = 5$ en el código.
- **Versión con bug:** el programador escribió $\mu = 4$ por error de transcripción.

Ambas pasan la verificación estructural sin problemas.

Resultados de $n_R = 30$ réplicas con $T = 2000$ min:

Métrica	Analítico	$\bar{Y}_{30}$	IC 95 %	Diagnóstico
Versión correcta ($\mu = 5$, $\rho = 0{,}60$)				
W_q (min)	0,300	0,308	[0,293; 0,323]	Validado ✓
L_q (cl.)	0,900	0,924	[0,877; 0,971]	Validado ✓
ρ	0,600	0,598	[0,596; 0,600]	Validado ✓
Versión con bug ($\mu = 4$, $\rho_{\text{aparente}} = 0{,}75$)				
W_q (min)	0,300	0,762	[0,701; 0,823]	Rechazado
ρ	0,600	0,749	[0,745; 0,753]	Rechazado

Ejemplo 2 — Lectura del bug a partir del IC



Diagnóstico mecánico:

- $\hat{\rho}_{\text{bug}} \approx 0{,}749 \approx 3/4$.
- Si la tasa de llegada es la documentada ($\lambda = 3$), la tasa de servicio efectiva es $\hat{\mu} = \lambda/\hat{\rho} \approx 4$.
- Apunta directamente al parámetro mal cargado.

Verificación pasó, validación detectó

El código *funcionaba* correctamente — solo con el parámetro equivocado. Trazas, conservación y pruebas degeneradas habrían pasado idénticas. Sin la comparación contra una referencia analítica, el bug habría llegado a producción.

Resultado del Ejemplo 2

Errores en **parámetros** no se detectan por verificación (todo pasa); se detectan por **validación** (los IC no contienen la referencia). Por eso ambos procesos son obligatorios.

Sistema

Una celda de manufactura recibe piezas en proceso de Poisson con $\lambda = 8$ pzas/h. El servicio tiene dos etapas (carga y mecanizado) exponenciales con $\mu_{\text{fase}} = 10$ op/h cada una:

$$S \sim E_2(10), \quad \mathbb{E}[S] = 0{,}2 \text{ h}, \quad \mathrm{CV}_S = 1/\sqrt{2}$$

Parámetros derivados:

$$\rho = 8 \times 0{,}2 = 0{,}80$$

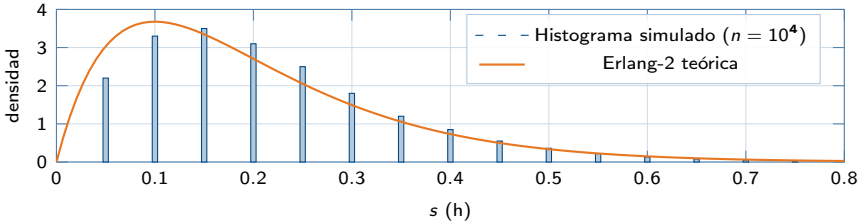
$$\mathbb{E}[S^2] = \mathrm{Var}[S] + \mathbb{E}[S]^2 = 0{,}02 + 0{,}04 = 0{,}06$$

Modelo conceptual: cola $M/E_2/1$ (Poisson + Erlang-2 + 1 servidor). Aplicaremos los **6 pasos** del proceso integrado de V&V.

Ejemplo 3 — Verificación

Prueba	Esperado	Resultado
Tráfico extremo $\lambda = 9{,}9$	$L_q \to \infty$ en $T = 500$ h	$L_q > 200$ pzas ✓
Simplificación: E_2 vs. $S_1 + S_2$	$\mathbb{E}[S] = 0{,}2$, Var $= 0{,}02$, asimetría $\sqrt{2} \approx 1{,}41$	Coincide al 4to decimal ✓
Conservación, $T = 2000$ h	$\approx 16\,000$ llegadas	$16\,023 = 16\,023 + 0$ ✓
Aserciones	Ninguna falla	0 violaciones en 10^7 eventos ✓

Distribución empírica del servicio S vs. Erlang-2



Lectura

El generador de tiempos de servicio implementa correctamente $S = S_1 + S_2$ con $S_i \sim \mathrm{Exp}(10)$. La verificación está cerrada.

Ejemplo 3 — Validación con Pollaczek–Khinchine

P-K para $M/G/1$

$$W_q^{\text{P-K}} = \frac{\lambda\,\mathbb{E}[S^2]}{2(1-\rho)}$$

Cálculo:

$$W_q^{\text{P-K}} = \frac{8 \times 0{,}06}{2 \times 0{,}2} = \frac{0{,}48}{0{,}4} = 1{,}20 \text{ h}$$

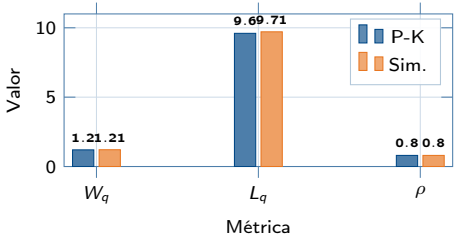
Comparación con $M/M/1$ equivalente ($\mathbb{E}[S] = 0{,}2$):

$$W_q^{M/M/1} = \frac{0{,}8}{5 \times 0{,}2} = 1{,}60 \text{ h}$$

$\Rightarrow$ Erlang-2 reduce W_q en 25 % gracias a la menor variabilidad del servicio.

Resultados de simulación
($n_R = 50$ réplicas, $T = 2000$ h)

Métrica	P-K	Sim.	IC 95 %
W_q (h)	1,200	1,214	[1,176; 1,252]
L_q (pzas)	9,600	9,712	[9,408; 10,016]
ρ	0,800	0,799	[0,795; 0,803]



Validación operacional

Todos los IC al 95 % contienen el valor de referencia. Error relativo $< 1{,}2$ % en las tres métricas. **El modelo está validado.**

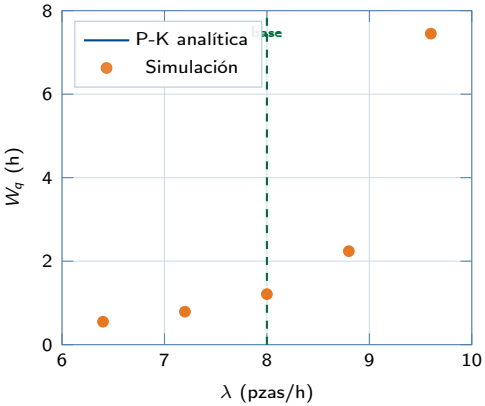
Ejemplo 3 — Análisis de sensibilidad sobre λ

λ	ρ	$W_q^{\text{P-K}}$	$\bar{W}_q^{\text{sim}}$	Dir.
6,4	0,64	0,53	0,55	✓
7,2	0,72	0,77	0,79	✓
8,0	0,80	1,20	1,21	—
8,8	0,88	2,20	2,24	✓
9,6	0,96	7,20	7,45	✓

Tres lecturas:

- Dirección correcta (W_q aumenta con λ). ✓
- No-linealidad: $+20\,\%$ en λ produce $+500\,\%$ en W_q — coherente con teoría de colas cerca de saturación. ✓
- Error relativo $< 4\,\%$ en todos los puntos. ✓

W_q vs. λ (sensibilidad)



V&V cerrada para el Ejemplo 3

Verificación: conservación, trazas y pruebas extremas correctas. Validación: W_q, L_q, ρ caen dentro del IC 95 % de la referencia P-K (error rel. $< 1{,}2\,\%$). Sensibilidad: dirección, magnitud y monotonía coherentes. **El modelo gana credibilidad para análisis fuera del alcance analítico** (percentiles, prioridades, múltiples colas).

El conjunto de evidencias

Validar un modelo no es un único test, sino un *conjunto* de evidencias:

Sesión 1: Verificación	**Sesión 2: Validación**	**Aporta evidencia sobre**
Debugging, aserciones	—	Código sin defectos puntuales
Trazas + invariantes	—	Código respeta lógica del modelo
Pruebas unitarias	—	Casos límite y analíticos correctos
—	*Face validity*	Modelo razonable a juicio experto
—	Validación estructural	Lógica del modelo refleja el sistema
—	χ^2, KS sobre datos	Distribuciones de *inputs* correctas
—	Calibración (división de muestra)	Parámetros ajustados generalizan
—	I/O con t pareada	Salidas del modelo $\approx$ realidad
—	Test de Turing	Salidas indistinguibles para expertos
—	Análisis de sensibilidad	Modelo responde *como debe* a cambios

Definición operativa de modelo válido

Un modelo es "válido" cuando la evidencia acumulada en estos planos es suficiente para que las decisiones que se tomen con él tengan **respaldo razonable**.

Errores frecuentes en V&V

Error	Síntoma	Cómo evitarlo
Validar con los mismos datos usados para calibrar	Error relativo $< 1\,\%$ "demasiado bueno"	Partir el histórico en $\sim 70/30$ — calibrar / validar
Validar solo la media	La media coincide pero los percentiles no	Comparar también $W_q^{(95)}$, varianza y forma de la distribución
Olvidar el período de calentamiento	Sesgo persistente en los IC	Descartar el transiente (ver T3.4)
Usar pocas réplicas	IC muy ancho, no decidible	Aumentar n_R hasta que el IC sea $<$ 5 % del valor estimado
Validar solo en el punto base	Modelo válido localmente, falso fuera	Análisis de sensibilidad con varios escenarios
Confundir significancia estadística con práctica	"Rechazar H_0" por diferencia trivial	Acompañar la prueba con error relativo y juicio del experto

Anti-patrón típico

"El modelo coincide con los datos del año pasado, así que está validado para proyectar el próximo". La validez *histórica* no garantiza la validez *prospectiva* si el sistema cambió (políticas, demanda, tecnología).

Mensajes finales

1. **Validación** (¿modelo correcto?) es **conceptualmente primaria**; la verificación (¿correctamente construido?) es indispensable pero secundaria.
2. Cada técnica formal vista hoy responde a **una pregunta distinta**; no se sustituyen entre sí.
3. Reportar siempre **intervalos de confianza**, no solo p-valores; distinguir **significancia estadística** de **relevancia operacional**.
4. Calibrar y validar con **datos independientes** (división de muestra).
5. El **análisis de sensibilidad** cierra el ciclo: dirección, magnitud y monotonía deben ser consistentes con la teoría.
6. V&V es **iterativa**: cada cambio en el modelo exige re-verificar y re-validar.

Mensaje para llevar

Un modelo verificado pero no validado es código limpio que responde la pregunta equivocada. Un modelo validado pero no verificado es una opinión correcta sobre un sistema mal programado. **Hacen falta los dos.**

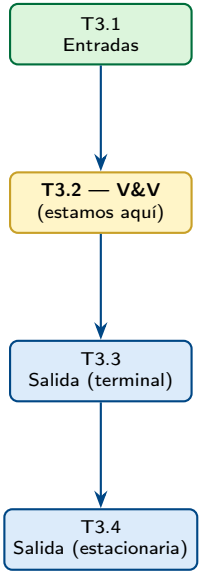
Próximo tema — T3.3: Análisis de salida (terminales)

De la V&V al uso del modelo

Con el modelo *verificado* y *validado*, podemos finalmente usarlo para responder preguntas que la teoría no resuelve. Pero antes de reportar *cualquier* número, debemos saber **cómo promediar las réplicas**.

En T3.3 veremos:

- Simulaciones *terminales* vs. estacionarias.
- Réplicas independientes y construcción de IC.
- Determinación del número de réplicas necesarias.
- Reporte de métricas con incertidumbre.



Continuidad

Referencias

- Banks, J., Carson, J. S., Nelson, B. L., Nicol, D. M. (2010). *Discrete-Event System Simulation* (5th ed.). Pearson. Caps. 1, 10.
- Law, A. M. (2015). *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill. Cap. 5.
- Kelton, W. D., Sadowski, R. P., Zupick, N. B. (2015). *Simulation with Arena* (6th ed.). McGraw-Hill.
- Ross, S. M. (2013). *Simulation* (5th ed.). Academic Press.
- Sargent, R. G. (2013). Verification and Validation of Simulation Models. *Journal of Simulation*, 7(1), 12–24.
- Schruben, L. W. (1980). Establishing the Credibility of Simulations. *Simulation*, 34(3), 101–105.
- Naylor, T. H., Finger, J. M. (1967). Verification of Computer Simulation Models. *Management Science*, 14(2), 92–101.
- Balci, O. (1997). Verification, Validation and Accreditation of Simulation Models. *Proceedings of the 1997 Winter Simulation Conference*.
- Lilliefors, H. W. (1969). On the KS Test for the Exponential Distribution with Mean Unknown. *JASA*, 64(325), 387–389.

¿Preguntas?

Verificación y Validación
en Simulación de Eventos Discretos